MULUNGUSHI UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

NAME: KAZEMBE SING'AMBWA

S/N: 202400114

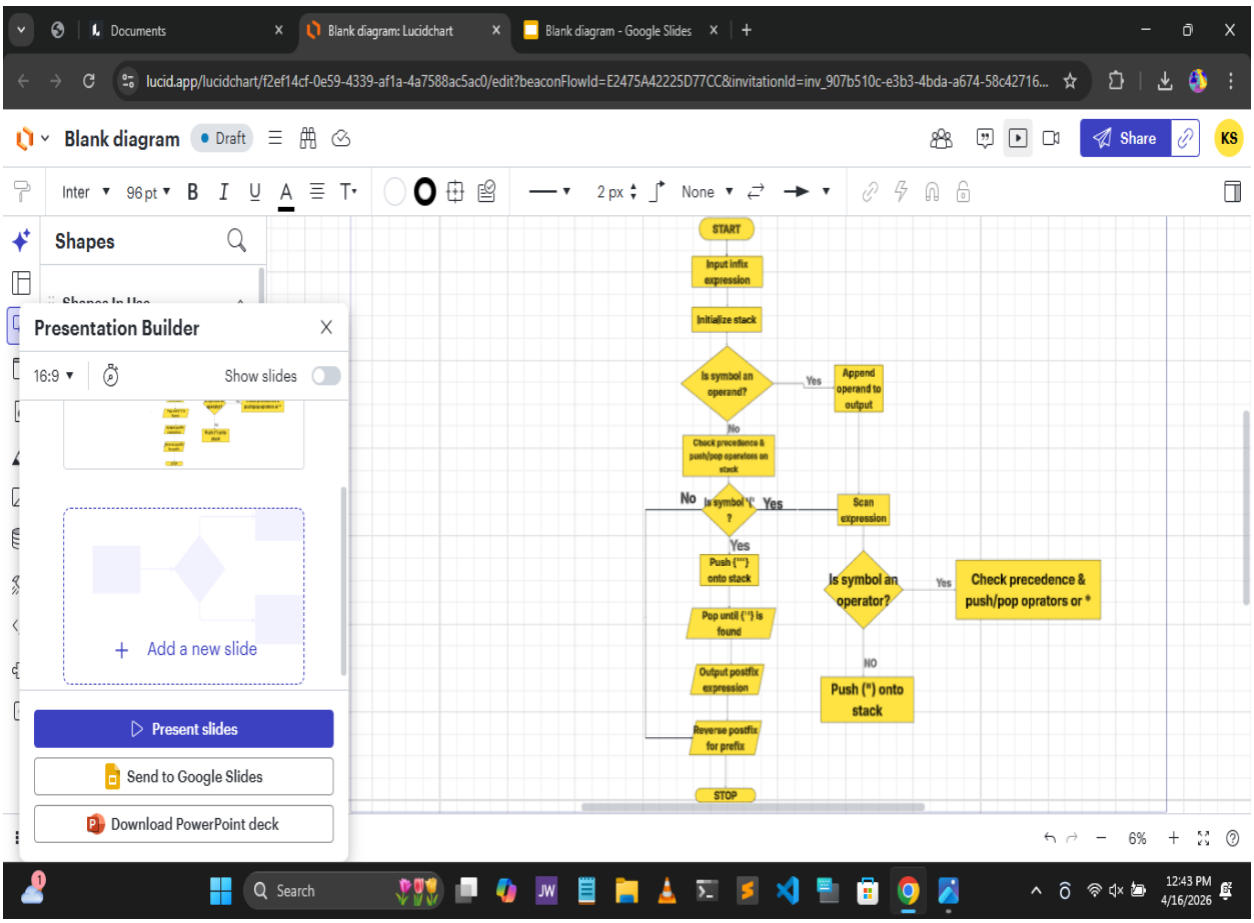
P.O.S: SCIENCE IN COMPUTER SCIENCE

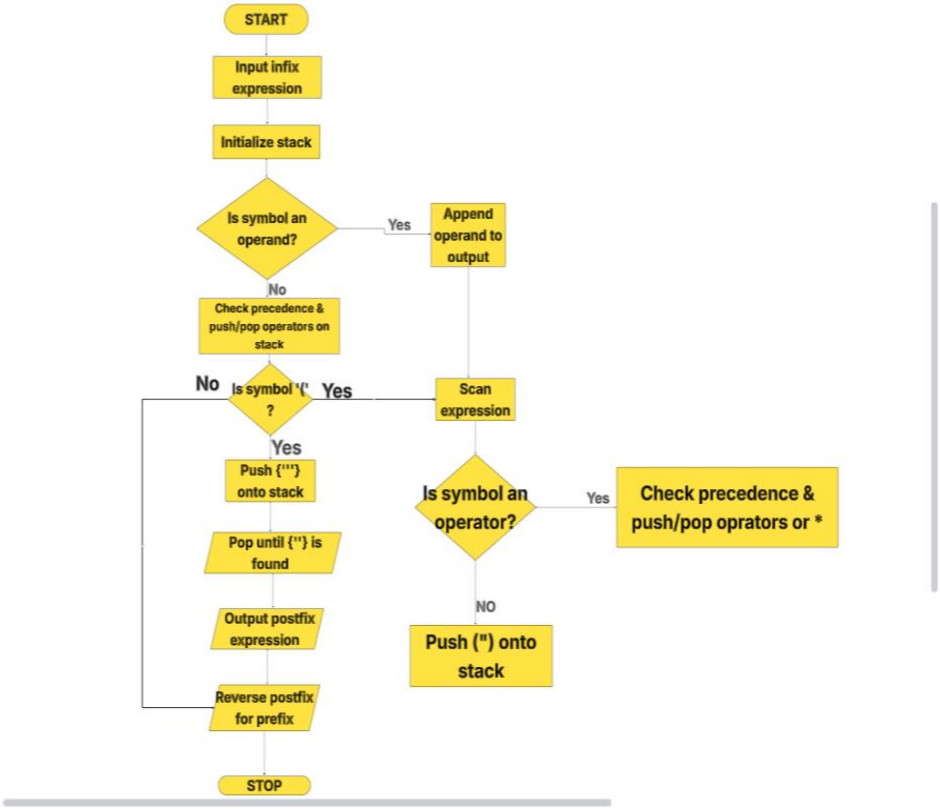
COURSE CODE: ICT 202

ASSIGNMENT.

This is a simple program that converts an infix expression into postfix and prefix. Below are the assignments visuals:

WORKFLOW:





2. PSEUDO CODE:

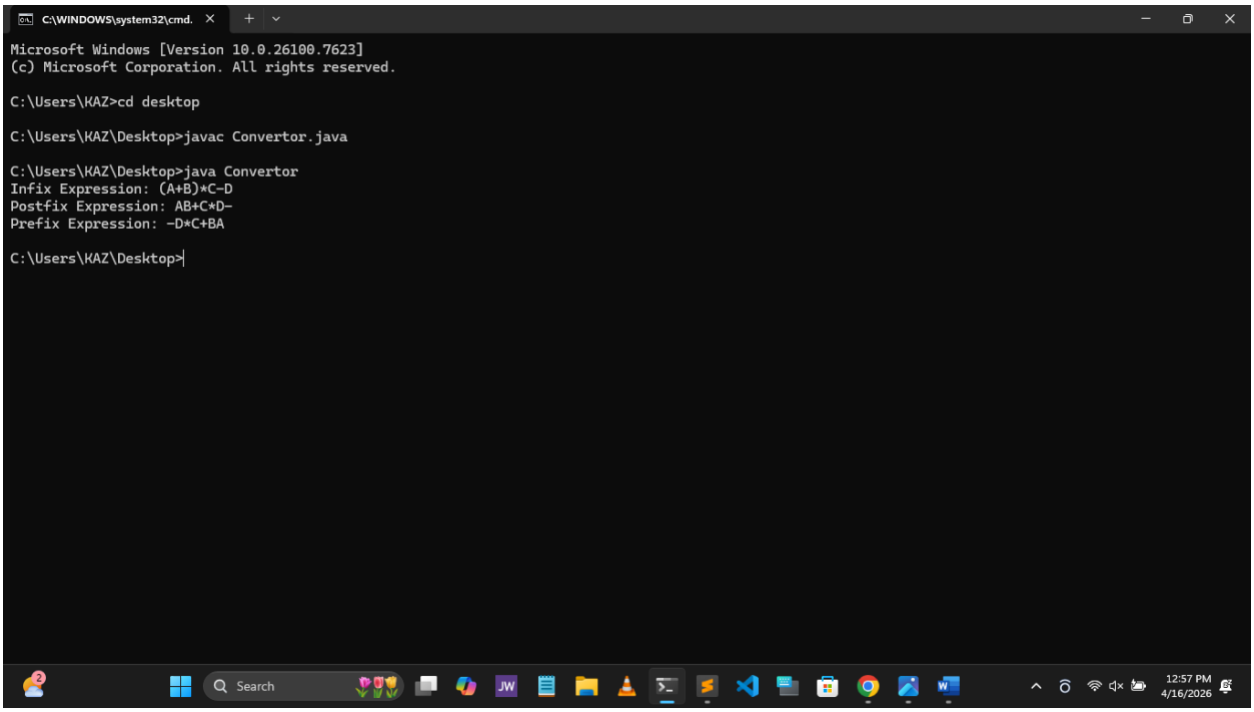
```
BEGIN
   INPUT infix_expression
   INITIALIZE empty stack
   INITIALIZE empty output_string

   FOR each symbol in infix_expression DO
      IF symbol is operand THEN
         APPEND symbol to output_string
      ELSE IF symbol is '(' THEN
         PUSH '(' onto stack
```

```
        ELSE IF symbol is ')' THEN
            WHILE top of stack != '(' DO
                POP operator from stack
                APPEND operator to output_string
            END WHILE
            POP '(' from stack
        ELSE IF symbol is operator THEN
            WHILE stack is not empty AND precedence(top of stack) >=
precedence(symbol) DO
                POP operator from stack
                APPEND operator to output_string
            END WHILE
            PUSH symbol onto stack
        END IF
    END FOR

    WHILE stack is not empty DO
        POP operator from stack
        APPEND operator to output_string
    END WHILE

    OUTPUT output_string as postfix_expression
    REVERSE postfix_expression to get prefix_expression
END
```

3. JAVA CODE:

```
import java.util.Stack;

public class Convertor {

    // Function to check precedence of operators
    private static int precedence(char operator) {
        switch (operator) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            case '^':
                return 3;
        }
        return -1;
    }

    // Function to convert infix to postfix
    public static String infixToPostfix(String infix) {
        Stack<Character> stack = new Stack<>();
        StringBuilder output = new StringBuilder();
```

```
        for (int i = 0; i < infix.length(); i++) {
            char symbol = infix.charAt(i);

            // If operand, add to output
            if (Character.isLetterOrDigit(symbol)) {
                output.append(symbol);
            }
            // If '(', push to stack
            else if (symbol == '(') {
                stack.push(symbol);
            }
            // If ')', pop until '('
            else if (symbol == ')') {
                while (!stack.isEmpty() && stack.peek() != '(') {
                    output.append(stack.pop());
                }
                stack.pop(); // remove '('
            }
            // If operator
            else {
                while (!stack.isEmpty() && precedence(stack.peek()) >=
precedence(symbol)) {
                    output.append(stack.pop());
                }
                stack.push(symbol);
            }
```

```
        }

        // Pop remaining operators
        while (!stack.isEmpty()) {
            output.append(stack.pop());
        }

        return output.toString();
    }

    // Function to reverse postfix to get prefix
    public static String postfixToPrefix(String postfix) {
        return new StringBuilder(postfix).reverse().toString();
    }

    public static void main(String[] args) {
        String infixExpression = "(A+B)*C-D";
        System.out.println("Infix Expression: " + infixExpression);

        String postfix = infixToPostfix(infixExpression);
        System.out.println("Postfix Expression: " + postfix);

        String prefix = postfixToPrefix(postfix);
        System.out.println("Prefix Expression: " + prefix);
    }
}
```

OUTPUT OF JAVA PROGRAM: